

# Technical Architecture Document:

## LedgerAI

**Document Status:** Final

**Target Audience:** Engineering Team (Backend, Frontend, AI/ML Engineers)

**Project Scope:** 1-Week Hackathon MVP & Future Production Roadmap

### 1. Architectural Goals

- **Modularity:** Decouple AI reasoning from deterministic execution (e.g., mathematical calculations).
- **Explainability:** Every autonomous decision must generate a traceable reasoning path ("Glass Box" transparency).
- **Fault Tolerance:** The system must gracefully handle LLM hallucinations, tool execution failures, and API timeouts.
- **Extensibility:** New agents (e.g., Payroll, Treasury) must be additive without requiring rewrites of the core orchestration loop.
- **Deterministic Reliability:** LLMs must never perform native arithmetic. All calculations must occur within isolated Python sandboxes.
- **Regulatory Compliance:** The system must natively support immutable audit logging and role-based human-in-the-loop (HITL) authorization.

### 2. Design Principles

- **Separation of Responsibilities:** Agents are scoped strictly to their domains. The Bookkeeper categorizes; the Audit Agent verifies. They do not share prompts.
- **Shared State (Blackboard Pattern):** Agents do not pass unstructured text directly to one another. They read from and write structured JSON to a centralized, schema-validated Blackboard.
- **Retrieval Before Generation (RAG-First):** Agents cannot rely on foundational model weights for tax laws. All compliance checks must query the vector database first.
- **Stateless APIs / Stateful Orchestration:** The backend REST API is stateless, but the orchestration engine (LangGraph) maintains immutable checkpoints of the graph state in PostgreSQL.

### 3. Overall System Architecture

flowchart TD

%% Client Tier

subgraph Client\_Tier [Client Tier]

WebUI[Next.js Frontend]

end

```

%% API Gateway & Backend
subgraph API_Tier [FastAPI Backend]
  Auth[Auth & JWT]
  Router[API Router]
  SSE[SSE Streamer]
end

%% Multi-Agent Orchestration (LangGraph)
subgraph Multi_Agent_Tier [Orchestration & Multi-Agent Layer]
  Orchestrator[Partner Agent / Router]
  Blackboard[(Blackboard / State Graph)]
  Bookkeeper[Bookkeeper Agent]
  GST[GST Specialist]
  Auditor[Audit Agent]
  Analyst[Financial Analyst]
  Compliance[Compliance Agent]
end

%% Tool & Execution Layer
subgraph Tool_Tier [Tool & Execution Layer]
  PythonEnv[Python Sandbox]
  DocParser[OCR / Document Parser]
  GovAPI[Gov / Bank APIs]
end

%% Data Layer
subgraph Data_Tier [Data & Memory Layer]
  PG[(PostgreSQL - App DB)]
  VectorDB[(pgvector - RAG Memory)]
  S3[(Object Storage)]
end

%% Connections
WebUI <-->|REST / SSE| Router
WebUI <--> Auth
Router <--> Orchestrator
Router --> SSE

Orchestrator <--> Blackboard
Blackboard <--> Bookkeeper
Blackboard <--> GST
Blackboard <--> Auditor
Blackboard <--> Analyst

```

Blackboard <--> Compliance

Bookkeeper --> DocParser

Bookkeeper --> PythonEnv

GST --> VectorDB

GST --> GovAPI

Auditor --> PythonEnv

Orchestrator --> PG

DocParser --> S3

## 4. Complete Request Lifecycle

1. **Upload:** User uploads an invoice PDF via the React frontend.
2. **Ingestion:** The FastAPI backend receives the file, uploads it to S3, and triggers the Document Processing Service.
3. **Extraction:** The OCR tool extracts raw text and layout data.
4. **Orchestration Initialization:** The Partner Agent (LangGraph entry node) initializes a new state dictionary (the Blackboard) with the raw data.
5. **Task Planning:** The Partner Agent analyzes the request and routes execution to the Bookkeeper Agent and GST Agent in parallel.
6. **Agent Execution:**
  - The Bookkeeper drafts the journal entry and posts it to the Blackboard.
  - The GST Agent queries the RAG database for specific rules and posts the tax implications.
7. **Verification:** The Audit Agent reads the Blackboard, cross-checks the journal entry against the GST rules, and tests the math via the Python Sandbox.
8. **HITL Resolution (if needed):** If the Auditor flags a missing compliance document, the graph pauses, saves its state to PostgreSQL, and pushes an SSE notification to the frontend. The user provides input, and the graph resumes.
9. **Finalization:** The Partner Agent compiles the verified Blackboard state, commits the transaction to the App DB, and generates an executive summary.

## 5. Frontend Architecture

**Tech Stack:** Next.js (App Router), React, Tailwind CSS, Recharts, Zustand (State).

- **Dashboard:** Displays real-time KPIs (Cash runway, pending approvals).
- **The "Glass Box":** A specialized timeline component that visualizes the multi-agent execution graph. It subscribes to a Server-Sent Events (SSE) endpoint to display agent thoughts, RAG citations, and intermediate JSON payloads in real-time.
- **Approval Dialogs:** Renders dynamic forms based on the HITL pause state (e.g., asking for a specific GSTIN or LUT number).

## 6. Backend Architecture

**Tech Stack:** Python, FastAPI, SQLAlchemy, Pydantic, Uvicorn.

- **API Gateway:** Handles JWT authentication and routes requests.
- **Stateful Workers:** For the MVP, LangGraph execution runs via asynchronous background tasks within FastAPI. (*Future: Move to Celery/Temporal for distributed task queuing*).
- **Dependency Injection:** Database sessions and LLM clients are injected into the API routes for testability.
- **Streaming:** Utilizes StreamingResponse for SSE to push LangGraph state updates to the frontend instantly.

## 7. Multi-Agent Architecture

For the MVP, agents are constructed as nodes within a LangGraph state machine.

### Partner Agent (Orchestrator)

- **Purpose:** User intent routing and final summarization.
- **Inputs:** User query, overall Blackboard state.
- **Outputs:** Directed edge routing (which agent runs next) or final user report.

### Bookkeeper Agent

- **Purpose:** Data extraction and ledger drafting.
- **Inputs:** OCR text, previous transaction history (via Vector DB).
- **Outputs:** Proposed JSON journal entry (Debits/Credits).
- **Tools:** OCR Parser, Python Sandbox (for currency conversion).

### GST & Tax Specialist

- **Purpose:** Enforce indirect tax compliance.
- **Inputs:** Proposed journal entry, vendor metadata.
- **Outputs:** Tax flags, validated GST rates.
- **Tools:** RAG Retriever.
- **Prompt Strategy:** Specifically instructed to verify exemptions, such as zero-rated supplies to Special Economic Zones (SEZs) under Section 16 of the IGST Act<sup>1</sup>.

### Audit Agent

- **Purpose:** Implement the Disagree-or-Commit (DoC) verification protocol.
- **Inputs:** All prior agent outputs on the Blackboard.
- **Outputs:** Approved boolean, plus reasoning trace or discrepancy notes.
- **Tools:** Python Sandbox (to recalculate sums).

### Financial Analyst

- **Purpose:** Translate the ledger entry into business context.
- **Inputs:** Final verified ledger entry.
- **Outputs:** Narrative insights (e.g., "This SaaS renewal decreases cash runway by 2 days").

### Compliance & Fraud Agent

- **Purpose:** Anomaly detection.

- **Inputs:** Current transaction, historical transaction metadata.
- **Outputs:** Risk score (0.0 - 1.0) and alert flags.

(Future Agents: Payroll, Income Tax, Treasury, CFO).

## 8. Orchestrator

The Orchestrator is built using **LangGraph**. It defines the flow as a Directed Acyclic Graph (DAG) with cycles allowed for error correction.

- **Routing:** The Partner Agent uses OpenAI's Function Calling to return a structured routing decision.
- **Retries:** If the Audit Agent rejects the Bookkeeper's output, a conditional edge loops back to the Bookkeeper with the Auditor's critique appended to the Blackboard. The loop is capped at 3 iterations to prevent infinite token burns.

## 9. Shared Knowledge Layer (Blackboard)

**Purpose:** To prevent N-to-N conversational chaos. Agents do not chat; they mutate a shared Pydantic model.

**Schema Example:**

**JSON**

```
{
  "transaction_id": "txn_123",
  "raw_text": "...",
  "bookkeeper_draft": {
    "debits": [{"account": "Software", "amount": 1000}],
    "credits": [{"account": "AP", "amount": 1000}]
  },
  "gst_context": {
    "is_sez": true,
    "lut_present": false,
    "citations": ["Section 16(1)(b) IGST Act"]
  },
  "audit_status": {
    "approved": false,
    "reason": "SEZ supply flagged, but LUT number is missing. Awaiting human input."
  },
  "human_input_required": true
}
```

## 10. Agent Communication

- **Mechanism:** Synchronous sequential execution via LangGraph for the MVP. When an agent node executes, it receives the current Blackboard state, processes its specific sub-task, and returns a dictionary of state updates. LangGraph merges these updates into the global state.
- *(Future Architecture: Move to an asynchronous Pub/Sub event bus (Kafka/Redis) where agents act as independent microservices listening for Blackboard topic updates).*

## 11. RAG Architecture

**Why RAG?** Foundational models hallucinate specific tax code applicability.

**MVP Implementation:**

- **Knowledge Sources:** Indexed chunks of the IGST Act, CGST Rules, and SEZ guidelines.
- **Embeddings:** text-embedding-3-small (OpenAI).
- **Database:** PostgreSQL with pgvector.
- **Retrieval:** Hybrid search (TF-IDF keyword + Cosine Similarity vector search).
- **Real-World Example:** If an invoice indicates delivery to the "Noida SEZ", the GST Agent queries the Vector DB. The RAG pipeline retrieves the rule stating that supplies to an SEZ are zero-rated under Section 16 of the IGST Act, but specifically require a Letter of Undertaking (LUT) or an endorsed Bill of Export (BoE) to claim the input tax credit. The agent uses this exact citation to enforce compliance.

## 12. LLM Layer

- **Foundation Models:** OpenAI gpt-4o for the Partner, Audit, and GST agents (requires high reasoning). gpt-4o-mini for the Bookkeeper and Document Extraction (cost optimization).
- **Structured Outputs:** All LLM calls enforce strict JSON generation using Pydantic schemas via the OpenAI response\_format API.
- **Temperature:** Set to 0.0 for all accounting, GST, and Audit tasks to ensure deterministic logic. Set to 0.4 for the Financial Analyst for narrative generation.

## 13. Tool Layer

All tools are wrapped as Python functions accessible via LLM Tool Calling.

- **Python Sandbox (Critical):** Executes stringified mathematical formulas generated by the LLM using Python's asteval library. *Justification: Prevents LLM arithmetic hallucinations.*
- **OCR Tool:** Wraps AWS Textract / Azure Document Intelligence API.
- **Vector Search Tool:** Executes queries against pgvector.
- **Gov API Mock (MVP):** A mock function simulating GSTIN validation and LUT status checks.

## 14. Memory Architecture

- **Short-Term Memory:** The LangGraph StateGraph object, maintained in memory during a single request lifecycle.
- **Conversation Memory:** LangGraph Checkpointing saves the state to PostgreSQL at every step, allowing the graph to pause, persist, and resume later for HITL.
- **Long-Term Memory:** Finalized journal entries are written to the relational Transactions table. Semantic embeddings of company vendor categorizations are written to pgvector for future matching.

## 15. Data Architecture (ER Diagram)

```

erDiagram
    USERS ||--o{ COMPANIES : owns
    COMPANIES ||--o{ TRANSACTIONS : has
    COMPANIES ||--o{ DOCUMENTS : has
    TRANSACTIONS ||--o{ AUDIT_LOGS : generates
    DOCUMENTS ||--o{ TRANSACTIONS : substantiates

    TRANSACTIONS {
        uuid id
        string status
        jsonb debits
        jsonb credits
        timestamp created_at
    }

    AUDIT_LOGS {
        uuid id
        uuid transaction_id
        string agent_name
        string action
        jsonb reasoning_trace
    }

```

## 16. Document Processing Pipeline

1. **Upload:** Binary file saved to S3; UUID generated.
2. **OCR:** Cloud provider extracts raw text and bounding boxes.
3. **Entity Extraction:** gpt-4o-mini parses the text into a standard JSON schema (Vendor Name, Date, Total, Tax Amount, Line Items).

4. **Validation:** Schema validates types (e.g., Date must be ISO 8601).

## 17. Financial Processing Pipeline

The core logic executed by the agents:

1. **Categorization:** Match vendor to Chart of Accounts.
2. **GST Validation:** Check if Vendor is Domestic, Interstate, or SEZ. Apply SGST/CGST/IGST rules accordingly.
3. **Drafting:** Create double-entry bookkeeping array.
4. **Audit:** Ensure Debits == Credits and tax logic matches RAG citations.

## 18. Human-in-the-Loop (HITL)

When an agent triggers an escalation (e.g., "LUT missing for zero-rated supply"):

1. LangGraph hits an interrupt node.
2. The state is serialized and persisted to PostgreSQL using `langgraph.checkpoint.postgres`.
3. The frontend receives a "Needs Approval" flag.
4. The user inputs the data (e.g., the LUT number).
5. The backend calls `graph.stream(input, config={"configurable": {"thread_id": "123"}})` to resume exactly where it left off.

## 19. Reporting Engine

A separate FastAPI route aggregates finalized data from the Transactions table and passes it to the Financial Analyst Agent to generate narrative insights. The frontend renders the numerical data using Recharts (bar charts, line graphs for cash flow).

## 20. Glass Box Architecture

Transparency is a strict requirement for financial enterprise tools. The Next.js frontend subscribes to the backend SSE endpoint. As LangGraph yields node updates, the UI dynamically renders:

- Which agent is currently "thinking."
- The actual tools being called (e.g., Calling tool: `calculate_tax()`).
- The retrieved RAG citations supporting the decision.

## 21. Security

- **Authentication:** JWT via OAuth2 (FastAPI OAuth2PasswordBearer).
- **Authorization:** Role-Based Access Control (RBAC). Only users with "Admin" roles can trigger the HITL resume function.
- **Data Isolation:** Multi-tenant architecture. Every Postgres query and Vector search must include a `company_id` filter to prevent data leakage.

## 22. Deployment

- **Hackathon MVP:** `docker-compose.yml` orchestrating FastAPI, Next.js, and PostgreSQL/pgvector locally.



- **Production (Future):** Kubernetes cluster on AWS. Backend deployed on ECS/EKS. Database on AWS RDS for PostgreSQL. Documents on S3.

## 23. Folder Structure (MVP Monorepo)

```
ledger-ai/
├── frontend/ # Next.js App
│   ├── app/
│   ├── components/
│   └── store/ # Zustand state
├── backend/ # FastAPI App
│   ├── main.py
│   ├── api/ # REST Routes
│   ├── core/ # Config & Security
│   ├── agents/ # LangGraph Nodes (Partner, GST, Bookkeeper, etc.)
│   ├── tools/ # Python Sandbox, DB Search
│   ├── rag/ # pgvector indexing & retrieval
│   └── models/ # Pydantic & SQLAlchemy Models
└── docker-compose.yml
```

## 24. Tech Stack Summary

- **Frontend:** Next.js 14, TailwindCSS, Recharts.
- **Backend:** FastAPI, Python 3.11.
- **Orchestration:** LangGraph, LangChain core.
- **LLM:** OpenAI API (gpt-4o, gpt-4o-mini).
- **Database / Vector DB:** PostgreSQL with pgvector extension.
- **ORM:** SQLAlchemy 2.0.

## 25. MVP vs Future Architecture

| Component           | Hackathon MVP                            | Production Version (Future)                      |
|---------------------|------------------------------------------|--------------------------------------------------|
| Agent Orchestration | LangGraph (Synchronous cyclic execution) | Kafka/Redis Pub-Sub (Asynchronous microservices) |
| Tools               | Basic Python asteval sandbox             | Secure Dockerized code-execution environments    |
| Integrations        | Uploaded PDFs and manual entries         | Live Plaid/Stripe/ERP API syncs                  |

|                     |                                        |                                     |
|---------------------|----------------------------------------|-------------------------------------|
| <b>Vector DB</b>    | pgvector in local PostgreSQL container | Managed Pinecone or AWS OpenSearch  |
| <b>Agent Roster</b> | 5 core agents                          | 15+ agents (Payroll, Treasury, CFA) |

## 26. Risks & Mitigations

- LLM Math Hallucinations:**
  - Mitigation:* Strict prohibition of LLM arithmetic. All math is routed to the Python Sandbox tool.
- Infinite Agent Disagreement:**
  - Mitigation:* LangGraph recursion\_limit set to 5. If the Auditor and Bookkeeper cannot agree within 5 loops, state automatically defaults to HITL escalation.
- Context Window Bloat:**
  - Mitigation:* Agents only receive the Blackboard state, not the raw chat histories of other agents.

## 27. End-to-End Sequence Diagram

```
sequenceDiagram
    actor User
    participant UI as Next.js Frontend
    participant API as FastAPI Backend
    participant Partner as Partner Agent
    participant Blackboard as Shared State
    participant Bookkeeper as Bookkeeper Agent
    participant Auditor as Audit Agent
    participant Tools as Math/DB Tools

    User->>UI: Upload Invoice
    UI->>API: POST /api/upload
    API->>Partner: Init LangGraph State
    Partner->>Blackboard: Write Extracted Text
    Partner->>Bookkeeper: Route to Bookkeeper
    Bookkeeper->>Tools: OCR & Extract Entities
    Bookkeeper->>Blackboard: Post Draft Entry
    Partner->>Auditor: Route to Auditor
    Auditor->>Tools: Python Math Verification
    Auditor->>Blackboard: Post Approval Status
    Blackboard-->>API: Stream State (SSE)
```

API-->>UI: Update Glass Box Timeline  
Partner-->>API: Commit to DB  
API-->>UI: Display Final Ledger Report

## 28. Architecture Decision Records (ADR)

- **ADR 1: Why LangGraph over AutoGen/CrewAI?** LangGraph provides granular control over the state graph and native support for persisting checkpoints, which is absolutely mandatory for implementing our HITL pause/resume requirement. CrewAI is too rigid for complex conditional loops.
- **ADR 2: Why pgvector over Pinecone for the MVP?** To reduce the dependency footprint and simplify the docker-compose setup for a 1-week hackathon. PostgreSQL handles both relational transactional data and semantic vectors in one stack.
- **ADR 3: Why a Blackboard Architecture?** Direct peer-to-peer agent messaging creates exponential API cost scaling and loses track of the "source of truth." The Blackboard ensures all agents work from a single, auditable JSON schema, forcing structured interactions.

### Works cited

1. SEZ Supply ITC Refund: Officer Approval Guide - Legal Suvidha,  
<https://legalsuvidha.com/blog/approval-from-sez-officer-for-itc>
2. Special Economic Zone (SEZ): Meaning, Types, GST & Benefits - EximPe,  
<https://eximpe.com/blog/b2b/special-economic-zone-sez-meaning-types-benefits-gst-treatment-complete-guide-for-exporters-importers>